

HAPFLOW: the Taint Analysis Framework for OpenHarmony Apps

HAONAN CHEN, Beihang University, China

MINGYI ZHOU, Beihang University, China

YANJIE ZHAO, Huazhong University of Science and Technology, China

LI LI*, Beihang University, China

With the rapid growth of OpenHarmony, a new distributed operating system (OS), security and privacy issues have become major concerns. While taint analysis has proven effective in Android, OpenHarmony still lacks a comparable framework. Additionally, OpenHarmony's programming language, ArkTS, has several unique features compared to other languages like TypeScript and Java, including complex lifecycles, closure mechanisms, and a distinct API ecosystem. Directly adapting existing taint analysis tools for Android cannot achieve effectiveness in OpenHarmony applications.

To address this challenge, we propose HAPFLOW, a novel taint analysis framework tailored specifically for the OpenHarmony platform and ArkTS programs. This work presents: (1) an LLM-assisted method for identifying and categorizing source APIs for OpenHarmony, (2) a lifecycle and callback functions modeling approach that supports declarative UI in ArkTS, and (3) an IFDS-based taint propagation extension that accurately handles closure-induced cross-function data flows in OpenHarmony applications. Therefore, HAPFLOW enables precise inter-procedural data-flow analysis for OpenHarmony applications. We validate HAPFLOW's effectiveness on the HapBench benchmark, achieving 96.15% precision and 94.34% recall. Furthermore, HAPFLOW identifies 73 sensitive data leak flows across over 3,000 open-source projects with 8 false positives, completing over 98% of analyses within 10 seconds. These results demonstrate HAPFLOW's practicability and scalability for taint analysis in OpenHarmony applications.

CCS Concepts: • **Security and privacy** → **Software security engineering**.

Additional Key Words and Phrases: Taint analysis, Static analysis, OpenHarmony, ArkTS, Mobile security

1 Introduction

As a new operating system (OS), OpenHarmony enables seamless interactions among a wide array of device forms, from earphones and automobile head units to smart TVs and mobile phones. Through its cross-device collaborative architecture and high-performance kernel design, OpenHarmony has rapidly established a global ecosystem spanning smart homes, automotive systems, and the Internet of Things [13, 20, 24]. By the end of 2024, OpenHarmony had achieved a 19% market share in China, with over 1 billion ecosystem devices deployed and more than 15,000 native applications and atomic services launched. The platform's flagship development language, ArkTS, evolved from TypeScript and enhances application development through declarative UI paradigms, an enhanced type system, and sophisticated state management mechanisms.

*Corresponding author (lilicoding@ieee.org).

Authors' Contact Information: Haonan Chen, chenhaonan915@buaa.edu.cn, Beihang University, Beijing, China; Mingyi Zhou, zhoumingyi@buaa.edu.cn, Beihang University, Beijing, China; Yanjie Zhao, yanjie_zhao@hust.edu.cn, Huazhong University of Science and Technology, Wuhan, China; Li Li, lilicoding@ieee.org, Beihang University, Beijing, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART

<https://doi.org/10.1145/3793863>

However, as the OpenHarmony ecosystem continues to expand, **privacy leak and data security issues** are becoming increasingly prominent. In particular, malicious or poorly programmed behaviors within applications can lead to sensitive data leaks, posing serious threats to user privacy and security. Like other mobile platforms, OpenHarmony applications handle various types of sensitive information, including personal contacts, location data, device identifiers, and user preferences [4]. Without proper security analysis mechanisms, these applications may inadvertently or intentionally expose sensitive data through improper data handling practices, insecure network communications, or unauthorized access to system resources.

To address such privacy concerns, taint analysis [10, 41] has emerged as a cornerstone technique for data flow security analysis. This approach tracks the flow of sensitive data from sources (such as location services, contact lists, or device identifiers) to potentially risky sinks (such as network transmission points, file storage, or inter-process communication channels), enabling the detection of potential privacy violations¹. Taking the Android ecosystem as an example, taint analysis has proven highly effective, with various practical tools successfully identifying privacy vulnerabilities. Notable examples include FlowDroid [9], which performs static taint analysis on Android applications, ICCTA [19], which extends analysis to inter-component communication, and DroidSafe [14], which provides comprehensive security analysis. These tools have significantly improved Android's overall security posture by detecting privacy leak vulnerabilities through precise data flow tracking.

Despite the success of taint analysis in other platforms, OpenHarmony currently lacks such critical analysis frameworks. This gap represents a significant research challenge for several reasons. (1) First, existing taint analysis tools are primarily designed for Java and Kotlin applications in Android systems, making them incompatible with ArkTS's unique language features. (2) Second, OpenHarmony's distinctive application architecture, including its complex component lifecycle management, requires specialized modeling approaches that current tools cannot provide. (3) Third, the platform's novel programming paradigms, such as ArkUI's declarative UI components and OpenHarmony's unique API forms², introduce new call flow and data flow patterns that traditional analysis frameworks fail to capture.

Previous work ArkAnalyzer [12] provides a static analysis framework for OpenHarmony applications, offering parsing, intermediate representation, and control/data flow construction. However, ArkAnalyzer mainly focuses on semantic-level analysis, it does not support inter-procedural data-flow analysis, and thus cannot precisely track how sensitive data flows across multiple functions or components. ArkAnalyzer plays a role similar to Soot in the Android ecosystem: it serves primarily as a foundational static analysis infrastructure rather than a security analysis tool. Just as advanced taint analysis frameworks such as FlowDroid were developed on top of Soot [39], ArkAnalyzer provides the essential information for building more sophisticated inter-procedural data-flow analyses, but it does not possess such capabilities on its own. This technical gap leaves OpenHarmony applications vulnerable to unknown data leak risks and limits their adoption in security scenarios where privacy guarantees are essential.

To fill this gap, we propose HAPFlow, the first static taint analysis framework for native OpenHarmony applications, featuring a declarative lifecycle modeling strategy, source and sink identification for OpenHarmony, closure-aware ArkIR extensions, and an IFDS-based inter-procedural solver tailored to ArkTS semantics

The main contributions are as follows:

¹Throughout this paper, **data leak** refers specifically to potential source-to-sink flows detected by static taint analysis that represent security risks warranting further investigation, rather than confirmed instances of malicious data disclosure. Static analysis tools cannot definitively determine whether these flows result from legitimate user-authorized actions, necessary application functionality, or genuinely malicious behavior, and necessarily adopt a conservative approach that may flag both genuine privacy violations and benign application behaviors.

²**ArkUI**, OpenHarmony's native UI framework, employs a declarative programming model where developers describe the desired UI state rather than imperatively specifying how to achieve it, creating complex implicit data bindings and state propagation mechanisms that differ significantly from traditional imperative UI frameworks.

- We systematically identify and extract source and sink APIs for OpenHarmony applications, and categorize sources based on the distinct pathways through which sensitive data enters applications. Through analyzing over 10,000 APIs in the OpenHarmony ecosystem, we employ a combination of large language model analysis and manual verification to establish a validated set of source and sink APIs specific to OpenHarmony applications, providing the foundation for accurate taint analysis.
- We propose a novel taint analysis methodology tailored for the OpenHarmony system and the ArkTS language, which integrates two key innovations: lifecycle modeling to resolve complex implicit function calls in ArkTS (including event callback functions in the declarative UI); and an extension to the IFDS algorithm via a modified ArkIR that explicitly represents closure captured variables, thereby enabling precise inter-procedural taint analysis.
- We construct HapBench, a comprehensive benchmark dataset for evaluating taint analysis frameworks on OpenHarmony applications. HapBench covers seven typical privacy leak scenarios with 67 expert-crafted test cases. Through a comprehensive evaluation on this benchmark, HAPFLOW demonstrates strong performance with 96.15% precision and 94.34% recall, validating its effectiveness. When deployed on over 3,000 real-world projects, HAPFLOW discovers 73 sensitive data leak flows with 8 false positives, while maintaining efficient analysis speed, which confirms its practical utility and scalability for OpenHarmony taint analysis.

Artifact Availability. For the artifacts related to HAPFLOW, please refer to <https://figshare.com/s/8e5c07d1d6d6da88439b>.

2 Background and Motivation

2.1 Background

ArkTS. ArkTS is a programming language designed specifically for building distributed applications in the OpenHarmony. While maintaining the syntax style of TypeScript, it enhances the static type system and supports a high-performance runtime through the ArkCompiler, making it suitable for developing high-performance applications with consistent experiences across multiple devices. In addition, ArkTS adopts the declarative UI framework ArkUI, which, compared to traditional imperative and procedural UI development approaches, focuses more on the outcome of UI descriptions. By binding the UI to reactive data, the interface can automatically update in response to data changes, greatly simplifying the development process[6, 7].

Taint Analysis. Taint analysis is a program analysis technique used to track the flow of data from sources, such as user privacy information or other sensitive inputs, to sinks, which are sensitive operations within a program—such as log printing, SMS sending, or network transmission[10, 27]. By identifying these source-to-sink paths, taint analysis helps detect potential security risks like data leaks or unauthorized data usage. This technique is widely used in identifying security vulnerabilities and data privacy leaks. Depending on the level of precision required, taint analysis can be categorized into various types, including context-sensitive, path-sensitive, and field-sensitive analysis, which help improve the modeling of data dependencies in complex program structures. Static taint analysis is typically built on the program’s intermediate representation (IR) and utilizes structures such as the control flow graph (CFG) and the call graph (CG) to analyze the propagation paths of tainted data, allowing potential security risks to be identified without executing the program.

2.2 Motivation

2.2.1 Motivating Example. OpenHarmony applications may exhibit potential privacy risks through unintended data flows. Listing 1 presents a concise example demonstrating how such risks can manifest in an OpenHarmony application. The code defines a UI component `Index` that includes a lifecycle function `aboutToAppear` (lines 5–7). This function is invoked when the component is about to appear and assigns the user’s location data to the

component property info. In the build function, the ArkUI layout is constructed, including a button component with an anonymous click callback function (lines 10–12) that outputs the info property content to the application log. Consequently, when a user clicks the button after the component appears, this creates a potential data flow from a sensitive source (location data) to a potentially risky sink (log output), representing the type of privacy risk that warrants investigation through taint analysis.

```

1  @Entry
2  @Component
3  struct Index {
4    info: string = ''
5    aboutToAppear(): void {
6      this.info = JSON.stringify(geoLocationManager.getLastLocation()); //source
7    }
8    build() {
9      RelativeContainer() {
10        Button('press').onClick(() => {
11          hilog.info(0x0000, 'testTag', '%{public}s', this.info); //sink, leak
12        });
13      }
14      .height('100%')
15      .width('100%')
16    }
17  }

```

Listing 1. Example of sensitive data flow in OpenHarmony

2.2.2 Challenges. Currently, there is no dedicated taint analysis tool available for native OpenHarmony applications. Although the Android platform has developed mature tools such as FlowDroid, DroidSafe, and AmanDroid, these are primarily designed for applications written in Java or Kotlin. As a result, existing Android taint analysis tools cannot be directly applied to ArkTS applications, and the corresponding analysis techniques require targeted adaptation or redesign. As shown in Figure 1, due to the unique features of OpenHarmony applications, the challenges we face are mainly reflected in the following three aspects (C1–C3):

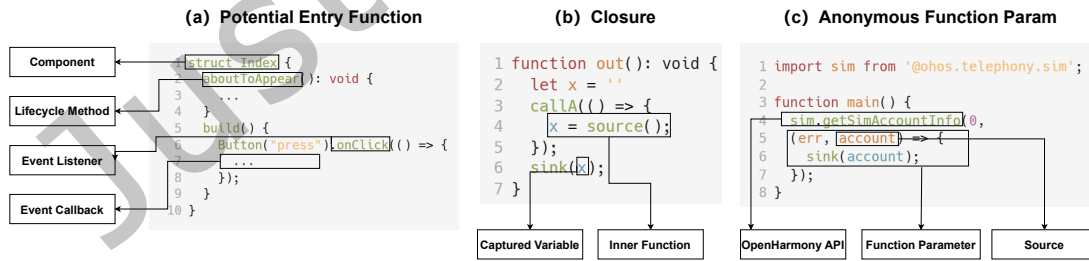


Fig. 1. Challenges in taint analysis of OpenHarmony applications

(C1) Complex Component Architecture and Lifecycle Management. The architecture of OpenHarmony applications is highly modular, typically composed of various component types, including `UIAbility`, `BackupExtensionAbility`, and `Struct`. Each component type possesses its own independent and complex lifecycle management mechanism, which controls behaviors and states at different stages, such as initialization,

execution, and destruction. For example, in the code snippet of Figure 1(a), the component Index is of type Struct, a lightweight view component structure unique to the ArkTS language in OpenHarmony. Its lifecycle function `aboutToAppear` is automatically invoked by the system when the component is about to appear on the interface, allowing developers to perform logic operations prior to interface rendering. These functions are not triggered by explicit function call statements in the program but are scheduled and executed by the framework at runtime based on the component's state. Consequently, such invocation relationships do not manifest in static code analysis, forming implicit edges in the analysis process.

Additionally, OpenHarmony employs the ArkUI framework for UI construction, which is centered around declarative UI programming. In this paradigm, developers declare component structures and behaviors, and the system automatically handles UI rendering and event binding based on state changes. Under this model, user interaction events such as button clicks and swipes are often embedded directly within component attributes as inline anonymous functions. For instance, developers bind callback handling logic such as `onClick` directly to button components using anonymous functions. While this inline callback handling approach improves development efficiency and code readability, it introduces significant challenges for data flow analysis. The scopes of anonymous functions are often unclear with complex variable capture relationships, and since these callback functions are not triggered through conventional function call chains but are dynamically bound and executed by the system based on user actions, taint analysis tools struggle to accurately identify these logical entry points and their propagation paths at the static level. Therefore, to comprehensively capture and model real data flows in OpenHarmony applications, it is necessary to design analysis mechanisms capable of identifying implicit function invocation relationships and adapting to the ArkUI programming paradigm to cover as many call chains as possible.

FlowDroid addresses similar lifecycle modeling challenges in Android by constructing a `dummymain`[9] method to simulate the application's global entry point, thereby linking lifecycle and callback invocations. However, this mechanism is valid for traditional View/XML architectures but not for declarative UI models. With the introduction of Jetpack Compose[3], callbacks are often implicitly bound within UI component structures. We conducted a small-scale experiment by implementing an Android application with identical data leakage logic, using both the traditional XML and imperative layout and the Declarative UI paradigm (as illustrated in Figure 2). Specifically, the traditional version (Listing 2) and the Jetpack Compose version (Listing 3) were compared. After packaging both into APKs for analysis by FlowDroid, the tool successfully detected the flow in the traditional XML version (Listing 2) but failed to detect it in the Declarative UI version (Listing 3). This outcome suggests a limitation in FlowDroid's current `dummymain` method and lifecycle modeling when analyzing applications built with Declarative UI frameworks. Given that ArkTS also utilizes a Declarative UI approach, new methodologies are necessary for accurately modeling its lifecycle for static taint analysis.

(C2) Impact of Closures on inter-procedural Data Flow Analysis. inter-procedural Finite Distributive Subset (IFDS) [32] is an efficient inter-procedural data flow analysis method widely used in taint analysis tools for Java and C/C++, such as FlowDroid [9] and Phasar [34]. However, JavaScript and TypeScript features such as dynamic typing and closures present additional challenges for static analysis [16, 31]. In contrast, ArkTS places greater emphasis on type safety and static features compared to JavaScript and TypeScript. It introduces a stricter static type-checking mechanism, requiring developers to explicitly declare variable types, which are then subject to compile-time type checks. Additionally, ArkTS restricts the use of the `any` type to minimize the impact of type uncertainty on program behavior [6]. These features constrain potential dynamic behaviors at the source code level, providing a solid semantic foundation for static data flow algorithms such as IFDS.

However, ArkTS retains closures, which pose residual challenges for data flow analysis. The closure mechanism allows an inner function to continue accessing external variables beyond its lexical scope, even when those variables are not passed through function parameters or explicitly referenced. While modern Java offers closure-like behavior through lambda expressions and anonymous inner classes, a critical distinction lies in its handling

1. Traditional Layout (XML + Imperative)

```

1 class MainActivity : AppCompatActivity() {
2     override fun onCreate(savedInstanceState:
      Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContentView(R.layout.activity_main)
5
6         findViewById<Button>(R.id.leakButton).
          setOnClickListener {
7             // Sensitive source: device ID
8             val tm = getSystemService(Context.
              TELEPHONY_SERVICE) as
              TelephonyManager
9             val deviceId = tm.deviceId
10            // Sink: SMS send
11            val sms = SmsManager.getDefault()
12            sms.sendTextMessage("1234567890",
              null, deviceId, null, null)
13        }
14    }
15 }

```

Listing 2. Traditional Layout Code (Imports removed)

2. Declarative UI (Jetpack Compose)

```

1 class MainActivity : ComponentActivity() {
2     override fun onCreate(savedInstanceState:
      Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             MyScreen()
6         }
7     }
8 }
9
10 @Composable
11 fun MyScreen() {
12     val ctx = LocalContext.current
13     Button(onClick = {
14         // Sensitive source: device ID
15         val tm = ctx.getSystemService(Context.
          TELEPHONY_SERVICE) as
          TelephonyManager
16         val deviceId = tm.deviceId
17         // Sink: SMS send
18         val sms = SmsManager.getDefault()
19         sms.sendTextMessage("1234567890", null,
          deviceId, null, null)
20     }) {
21         Text("Click Me")
22     }
23 }

```

Listing 3. Declarative UI Code (Imports removed)

Fig. 2. Comparison of the two Android code snippets.

of captured variables. In Java, any external variable accessed within a closure must be effectively final, meaning its value cannot be modified after being captured. This read-only access significantly simplifies static analysis, as the state of these captured variables remains immutable from the perspective of the closure.

ArkTS's closure semantics diverge significantly from this model, as captured external variables can be mutably modified within the inner function. As illustrated in the Figure 1(b), the anonymous function passed as a parameter to `callA` not only accesses the external variable `x` directly but also reassigns it using a source. This reassignment affects the enclosing function's state and ultimately causes a data leak at the sink point in the outer function. This mechanism is more similar to Kotlin's approach, which poses a significant challenge for existing Java-based taint analysis tools, as these tools perform poorly when handling similar issues in Kotlin [18].

Although similar closure-related data flow issues also exist in JavaScript, TypeScript, and Python, and these languages already have static taint analysis tools (such as CodeQL [1] and PySA [5]), these tools cannot be directly applied to ArkTS. CodeQL can analyze closures in JavaScript and TypeScript by recording captured-variable information in its variable tables, and PySA supports closure-related taint propagation in Python by abstracting captured variables as taint roots, their frontends and intermediate representations are fundamentally tied to

the syntax and execution semantics of those languages. As a result, they cannot accommodate ArkTS's unique AST structure or the declarative execution model of the ArkUI framework. In contrast, ArkAnalyzer provides a readable and extensible ArkIR tailored to ArkTS. Building on this foundation, we extend ArkIR to explicitly encode closure-capture semantics and integrate it with an IFDS-based data flow solver. This IR-centric design offers a more natural and effective solution than adapting existing general analysis tools to the OpenHarmony ecosystem.

(C3) Unique API Design of OpenHarmony. The API system of OpenHarmony differs significantly from that of Android, affecting both how sources and sinks are defined in taint analysis and altering the patterns of taint propagation. Compared to Android, OpenHarmony APIs are designed to be more dynamic and flexible, with a wider variety of methods to access sensitive data. In traditional taint analysis models, the return value of a sensitive API is typically treated as the source; however, this approach no longer covers all cases in the context of OpenHarmony. In certain OpenHarmony APIs, sensitive data is passed to callback functions through parameters, thereby indirectly achieving data transfer. The example of Figure 1(c) illustrates this complex data flow pattern: the API `sim.getSimAccountInfo` is used to retrieve users' SIM card account information, and its second parameter is an anonymous callback function. Within this callback, the second parameter serves as the actual source carrying sensitive information. In other words, the generation point of sensitive data does not explicitly appear at the API call site but is instead embedded within the parameters of the callback function. This implicit and dynamic data transmission mechanism significantly increases the complexity of static analysis and renders many Android-oriented taint analysis methods difficult to directly apply to OpenHarmony, imposing higher demands on taint propagation path modeling.

Summary

The essential taint analysis framework for detecting potential privacy issues is missing in OpenHarmony. Considering the complex component architecture and lifecycle, closure mechanism, and unique API set of OpenHarmony apps, we can borrow the workflow of existing frameworks in other platforms and elaborate techniques to resolve the challenge of taint analysis in OpenHarmony.

3 Methodology

3.1 Overview

The overall architecture of HAPFlow is illustrated in Figure 3. HAPFlow is built on the ArkAnalyzer static analysis framework [12], which provides the necessary prerequisite information such as basic program abstractions, control-flow graphs, and call graphs. While ArkAnalyzer focuses on general static analysis capabilities, HAPFlow extends it by modeling the application lifecycle and integrating an IFDS-based data-flow solver, enabling precise inter-procedural taint analysis. HAPFlow takes the ArkTS project source code along with the corresponding SDK files as input. Its output is the suspicious data flows that may lead to data leaks.

To address the technical challenge mentioned in § 2.2, HAPFlow's implementation consists of three main modules: **(1)** As shown in the top layer of Figure 3, it collects all potential entry functions within the project, including lifecycle methods and callback functions across various components. It can address the Potential Entry Function problem in analyzing OpenHarmony applications. These entry points are then connected and modeled to construct a virtual main function, *HarmonyMain*, which serves as the unified entry for data flow analysis. **(2)** The middle layer handles the configuration for the data flow analysis. In this stage, we first extract OpenHarmony API information from the ArkTS SDK, then apply large language models (LLMs) for automated identification and classification, and ultimately generate the required source/sink API configuration for the analysis. We use this module to resolve configuration issues related to the Anonymous Function Param problem (e.g., which APIs are sources, and which sources may involve anonymous function parameters). **(3)** In order to address the data flow

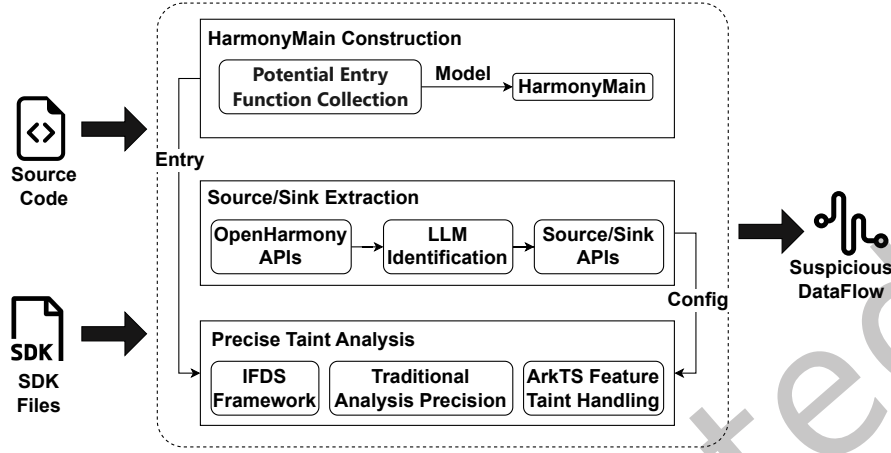


Fig. 3. Implementation overview of HAPFlow

propagation challenges introduced by the Anonymous Function Param problem and closures, the bottom layer contains HAPFlow's data flow analysis module, which is built on the IFDS algorithm framework. This module not only incorporates precision techniques commonly used in traditional taint analysis, such as context sensitivity, field sensitivity, flow sensitivity, and alias analysis, but also introduces tailored extensions and optimizations to address the specific language challenges of ArkTS.

3.2 HarmonyMain Construction

An OpenHarmony application consists of multiple components. UIAbility serves as the core component, primarily responsible for UI interaction and acting as the fundamental unit of system scheduling. BackupExtensionAbility handles application data backup and restoration. The struct component is used for defining custom UI structures, with its layout specified through the build method. These components are based on predefined class prototypes in the OpenHarmony system and implement specific lifecycle functions. The system automatically triggers these functions based on runtime requirements to manage component startup, suspension, resumption, or termination. As a result, lifecycle function calls are implicit and can serve as potential entry points for applications. Additionally, struct components typically include numerous callback functions that handle user interactions, and their callbacks are also implicitly invoked. As shown in Table 1, OpenHarmony supports a large number of lifecycle functions and callback functions, whose invocation paths are not explicitly presented, posing challenges for taint analysis.

To address this issue, the first step is to identify all lifecycle and callback functions within the application. Unlike FlowDroid's dummymain construction, which relies on predefined callback templates, HapFlow employs a more precise and dynamic lifecycle modeling strategy. It operates on the ArkIR that ArkAnalyzer generates from the declarative UI structure. Specifically, HapFlow iterates statement by statement through the ArkIR to identify calls to event-triggering functions and subsequently retrieves the corresponding callback functions from them. This design allows for precise modeling of application behavior, particularly for declarative UI components where callbacks are implicitly bound within the framework.

HAPFlow performs the extraction in follow stages. First, it conducts basic code analysis using ArkAnalyzer to obtain essential program abstractions. For lifecycle functions, HAPFlow traverses all methods defined in Ability components and @Component classes, matching their names against the predefined lifecycle method list (as shown in Table 1). For callback functions, it collects all build methods of @Component classes (i.e., UI layout functions), traverses their statements, and identifies invocations that match the callback names listed in Table 1. The corresponding callback functions are then extracted and integrated into the inter-procedural data flow analysis. We then construct a virtual entry function, *HarmonyMain* (illustrated in Figure 4), to explicitly invoke these implicit methods.

Since the execution order of these methods is non-deterministic, we design a loop structure to simulate all possible execution sequences, maximizing coverage of potential taint propagation paths. Within the loop body, each implicit function call is guarded by an *if* condition. However, during analysis, HAPFlow does not evaluate the actual values of these conditions; instead, it treats all branches equally. This ensures that all possible function invocation combinations are considered, allowing our taint analysis to capture the complete data flow within an OpenHarmony application.

Furthermore, *HarmonyMain* collects all static properties in the project along with their initialization expressions and converts them into three-address code stored in a method named *%static_init*. This method executes initialization in the same order as defined in the source code, ensuring that all static properties are in an initialized state during the analysis process. to detect the static property whose initial value is sources, *HarmonyMain* explicitly invokes all *%static_init* methods before entering the main analysis loop. Additionally, if a static property's initialization relies on function calls, cross-module references, or static blocks, our approach also analyzes and simulates their execution to ensure completeness.

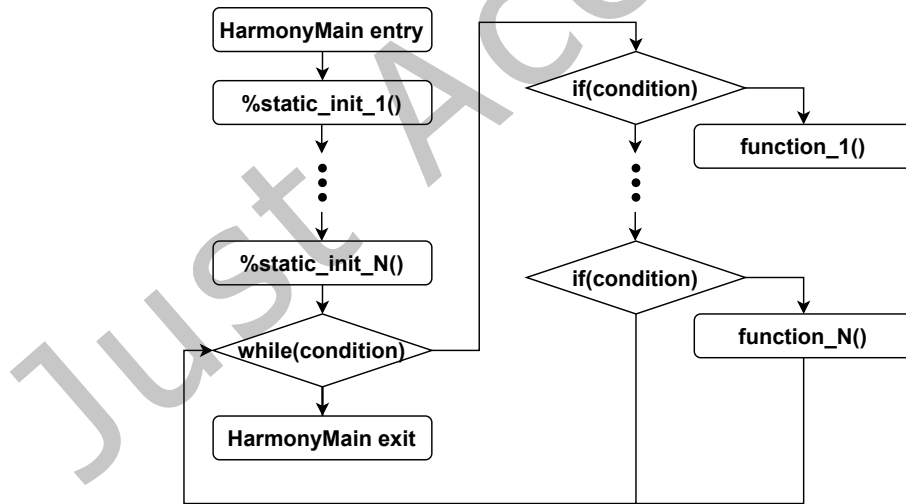


Fig. 4. *HarmonyMain* for implicit method invocation

The construction strategy of *HarmonyMain* provides a reusable blueprint for enhancing mobile static analysis tools. *HarmonyMain* explicitly models events triggered by the declarative UI paradigm, thereby enabling more

Table 1. Lifecycle methods and callback functions in OpenHarmony

Lifecycle Methods			Callback Functions
UIAbility	BackupExtensionAbility	struct	
onCreate	onBackup	aboutToAppear	onClick
onWindowStageCreate	onBackupEx	onDidBuild	onTouch
onWindowStageWillDestroy	onRestore	aboutToDisappear	onAppear
onWindowStageDestroy	onRestoreEx	onPageShow	onDisappear
onWindowStageRestore	onProcess	onPageHide	onDragStart
onDestroy		onBackPressed	onDragEnter
onForeground		aboutToReuse	onDragMove
onBackground		aboutToRecycle	onDragLeave
onContinue		onWillApplyTheme	onDrop
onNewWant			onKeyEvent
onDump			onFocus
onSaveState			onBlur
onShare			onHover
onPrepareToTerminate			onMouse
onPrepareToTerminateAsync			onAreaChange
onBackPressed			onVisibleAreaChange

accurate coverage of framework-driven execution paths. This design represents a significant step forward in modeling modern mobile architectures and is inherently applicable to environments where declarative UI is dominant, such as Android Jetpack Compose.

3.3 Source/Sink Extraction

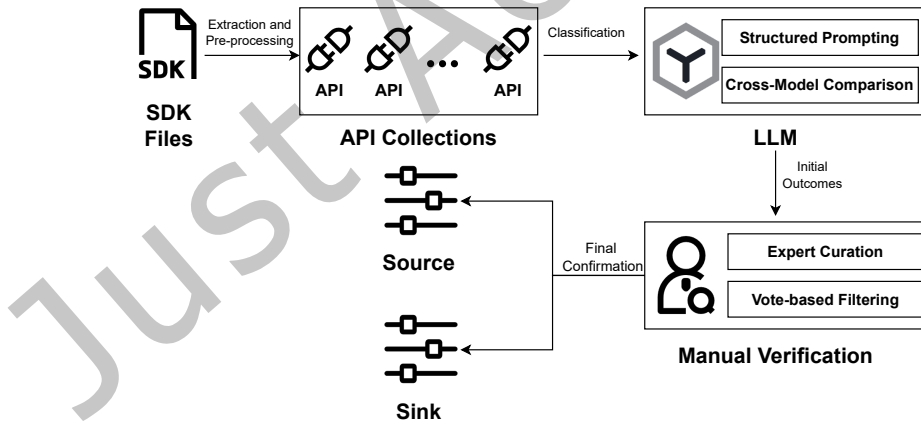


Fig. 5. LLM-assisted source and sink API identification process

To enable precise taint analysis for OpenHarmony applications, accurate identification of source and sink APIs within the OpenHarmony ecosystem is essential. This represents the second key component of our work. In this section, we propose a systematic method for extracting and cataloging Source and Sink APIs from the

OpenHarmony ecosystem, which combines automated extraction assisted by LLMs with validation by domain experts. The overall process is illustrated in Figure 5.

First, we extracted over 12,019 API definitions from the OpenHarmony SDK and performed an initial data cleaning and parsing process. This involved extracting key metadata for each API, including its module, namespace, class, name, parameters, return type, and associated documentation. This structured information was then provided to our LLM-based analysis pipeline. To ensure the accuracy and rigor of the LLM analysis, we constructed a detailed prompt that positioned the LLM as privacy security experts.

Table 8 presents the full prompt template used to classify APIs as Source, Sink, or None, including task objectives, input fields, output schema, and example annotations. To more accurately model how taint enters a program, this work further categorizes Source APIs into three types based on their entry mechanisms. First, `returnSource` refers to cases where sensitive data is obtained directly from a function’s return value, such as `@ohos.geoLocationManager.getLastLocation()`, whose return object contains device location information. Second, `paramSource` captures system-injected parameters in lifecycle functions, which not explicitly invoked by the developer but automatically supplied by the framework at runtime. A representative example is the `want` parameter in `@ohos.app.ability.UIAbility.onCreate(want: Want, launchParam: AbilityConstant.LaunchParam)`, which, in OpenHarmony, serves as the carrier for inter-component communication and may contain device-related metadata or contextual information. Third, `callbackSource` denotes scenarios where sensitive information is delivered through the argument of a callback function; for instance, in `@ohos.pasteboard.SystemPasteboard.getData(callback: AsyncCallback<PasteData>)`, the sensitive data is provided through the callback parameter. By explicitly distinguishing these entry pathways in the prompt design, the classification process more faithfully reflects framework-driven data flows, thereby enhancing the precision and real-world utility of static taint analysis.

To ensure the accuracy of the Source and Sink API sets extracted using the LLM, we performed validation via independent double-blind manual annotation by two human experts. The annotation criteria were established as follows: Source APIs were classified into three sensitivity levels: High (closely related to private information), Medium (potentially related), and None (unrelated); Sink APIs were classified into two leakage levels: Yes (confirmed to leak or exfiltrate data) and No (confirmed not to leak or exfiltrate data). We calculated Cohen’s Kappa (κ) coefficient to measure the inter-rater reliability between the two annotators. The results yielded a κ value of 0.8955 for Source APIs and 0.8601 for Sink APIs. According to established benchmarks, these values indicate high agreement, confirming the reliability and accuracy of our verification results. Disagreements were resolved through expert discussion to achieve a final consensus. The finalized dataset comprises 730 High Sensitivity Source APIs, 1336 Medium Sensitivity Source APIs, and 172 confirmed Sink APIs. It is important to note that the API count reflects distinct function signatures, including all relevant function overloads.

3.4 Precise Taint Analysis

HAPFlow implements an efficient taint analysis based on the IFDS analysis framework [30, 32]. Our implementation not only supports various traditional inter-procedural data flow analyses but also addresses the specific challenges outlined in § 2.2.2, thereby enabling precise taint analysis for OpenHarmony applications. The following sections will detail our approach.

3.4.1 Practical Precision Implementation in ArkTS Taint Analysis. HAPFlow implements the precision guarantees of the IFDS framework through practical integration with ArkAnalyzer. This section illustrates how HAPFlow preserves flow-, field-, context-, and alias-sensitivity in ArkTS-specific scenarios. We now present Figure 6 to illustrate these adaptation strategies in detail.

```

1 // (1) Flow Sensitivity
2 let x = source();
3 x = "sanitize";
4 sink(x); //no leak

1 // (2) Field Sensitivity
2 a.b = source();
3 sink(a.b); //leak
4 sink(a.c); //no leak

1 // (3) Context Sensitivity
2 function onCreate(): void {
3     let s = "ArkTS";
4     this.log(s)
5 }
6 function log(s: string) {
7     sink(s); //no leak
8 }
9 function unreachable() {
10    let s = source();
11    this.log(s);
12 }

1 // (4) Alias Sensitivity
2 function onCreate() {
3     let c = new User();
4     taint(c);
5     sink(c.info); //leak
6 }
7
8 function taint(a: User) {
9     sink(a.info); //no leak
10    let b = a;
11    b.info = source();
12 }

```

Fig. 6. Example code snippets demonstrating sensitivity requirements for taint analysis

Flow Sensitivity. HAPFLOW maintains a precise control-flow-based taint propagation mechanism. Each assignment updates the taint state of variables, ensuring that overwritten values no longer carry outdated taints. This prevents false positives caused by stale data states, as illustrated in Figure 6(1).

Field Sensitivity. HAPFLOW distinguishes taint information for each field node rather than treating an object as a whole. Independent field-level nodes enable accurate propagation tracking, as shown in Figure 6(2), where only `a.b` carries sensitive data. This design substantially improves precision in object-oriented taint tracking.

Context Sensitivity. Leveraging the inherent context-sensitivity of the IFDS framework, HAPFLOW performs call-site-specific data-flow tracking. This approach maintains independent data-flow contexts per call chain, preventing incorrect taint merging. As shown in Figure 6(3), this ensures different function calls do not share taint incorrectly, accurately ignoring unreachable data flow.

Alias Sensitivity. HAPFLOW implements specialized handling for cases where multiple references point to the same object. Alias relationships between objects often lead to implicit taint propagation across different variables. If left unaddressed, this could result in missing potential data leak risks. To mitigate this issue, HAPFLOW integrates ArkAnalyzer's alias analysis, employs a context-sensitive Andersen-style pointer analysis and addresses the challenges posed by TypeScript's dynamic features and OpenHarmony framework interactions through a unique ArkTS heap object model and a mixed context-sensitivity strategy. For example, in Figure 6(4), without alias analysis, the tool would fail to recognize that `c.info` should also be marked as tainted, leading to false negatives.

After incorporating alias analysis results, ArkAnalyzer can identify that the variables `a` and `b` in the `taint` function, as well as `c` in the `onCreate` function, all reference the same object. However, efficiently propagating taint across aliases remains a challenge. The most straightforward approach would be to propagate the taint to all alias objects' corresponding fields when `b.info` is marked, meaning both `a.info` and `c.info` would be tainted simultaneously. However, this approach has a drawback—within the IFDS framework, Normal Edge is limited to the current function scope. Directly propagating taint to local variables across function boundaries would significantly increase computational overhead and weaken the efficiency advantage of IFDS.

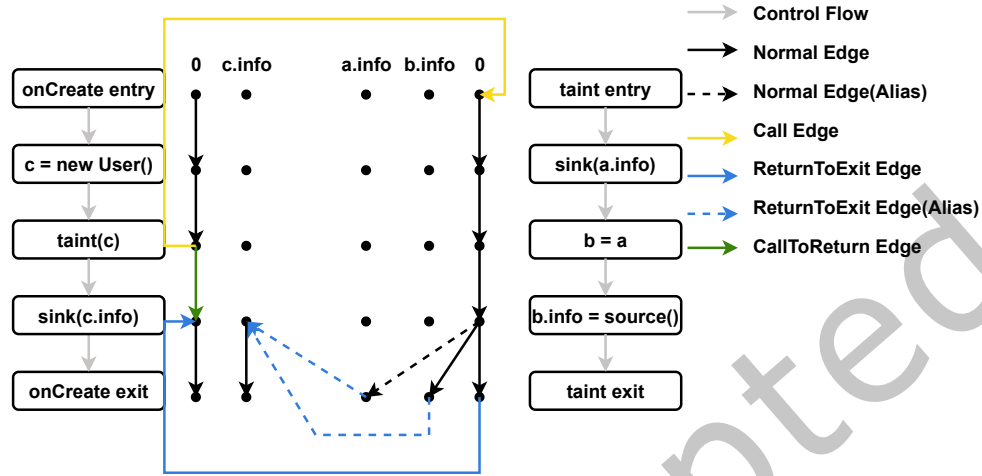


Fig. 7. Example of integrating alias analysis to HAPFLOW.

To address these challenges, HAPFLOW employs a precise strategy that integrates alias analysis with the IFDS framework, ensuring both efficiency and accuracy in taint propagation. The strategy can be divided into two main parts:

- **Intra-procedural handling:** Within a function, taint propagation and reassignment are both alias-aware. When a variable is marked as tainted, all its alias variables in the current function scope are also marked as tainted and propagated past the current statement. Conversely, when a variable is reassigned, the taint variable and all its alias variables is cleared.
- **Inter-procedural handling:** At the call site, no additional alias propagation is performed, since intra-procedural alias handling already ensures that the arguments are correctly tainted. Upon function return, HAPFLOW checks whether the callee modified any tainted variables that have alias counterparts in the caller. If such aliases exist, the taint is propagated back to the caller context, ensuring precise inter-procedural alias-aware taint tracking without excessive propagation.

As shown in Figure 7, when `b.info` is marked as tainted in the Normal Edge, the taint only propagates to all alias variables within the current function scope (e.g., `a.info`). This results in the addition of a new Normal Edge due to aliasing (represented by the black dashed arrow). Once the taint function completes and returns, at the ReturnToExit Edge, HAPFLOW checks whether the target function contains alias variables related to the tainted variable (e.g., `c.info`). If such aliases exist, the taint continues to propagate, forming a new ReturnToExit Edge due to aliasing (represented by the blue dashed arrow). This approach ensures precise control over alias taint propagation.

3.4.2 ArkTS Feature Taint Handling. As discussed in Motivation, the closure and anonymous function parameter features inherent to ArkTS present new challenges for taint analysis. Before processing the propagation of tainted

variables, we must first address issues related to closure-related function calls. However, the current ArkAnalyzer implementation of CHA/RTA-based call graph construction does not correctly handle these cases. Specifically, it treats both the function expression and all function-type parameters as possible callees regardless of whether the parameters are actually invoked inside the expression body. Therefore, we enhance the call resolution process to adapt IFDS analysis to ArkTS-specific function usage patterns.

DEFINITION 3.1 (ENHANCED CALL RESOLUTION). *Let c be a call site with invoke expression e and arguments $\{a_1, a_2, \dots, a_n\}$. The complete callee set of c is defined as:*

$$\text{Callees}(c) = \text{Callees}_{\text{expr}}(c) \cup \text{Callees}_{\text{param}}(c)$$

where

$$\text{Callees}_{\text{expr}}(c) = \begin{cases} \text{CG}(e) \setminus \{\text{ValueToArkMethod}(a_i) \mid a_i.\text{type} = \text{FunctionType}\}, & \text{if } e \text{ is a project-defined function call} \\ \text{ValueToArkMethod}(e), & \text{if } e \text{ is a function-type variable call} \\ \emptyset, & \text{if } e \text{ is an SDK function call} \end{cases}$$

and

$$\text{Callees}_{\text{param}}(c) = \begin{cases} \{\text{ValueToArkMethod}(a_i) \mid a_i.\text{type} = \text{FunctionType}\}, & \text{if } e \in \text{SDK} \\ \{\text{ValueToArkMethod}(a_i) \mid a_i.\text{type} = \text{FunctionType}, i \in \text{InvokedParams}(\text{Callees}_{\text{expr}})\}, & \text{otherwise} \end{cases}$$

Here:

- $\text{CG}(e)$ denotes the set of callees obtained from the call graph of ArkAnalyzer.
- $\text{ValueToMethod}(x)$ denotes the set of all functions that a function-type variable x may reference, as determined by alias and type analysis.
- $\text{InvokedParams}(\text{Callees}_{\text{expr}})$ is the index set of parameters actually invoked inside $\text{Callees}_{\text{expr}}$'s body.

The enhanced call resolution method refines ArkAnalyzer's original call graph lookup process through two coordinated analyses, as formally defined in Definition 3.1 and implemented in Algorithm 1. First, the expression-level resolution differentiates between project-defined functions, SDK functions, and function-type variables. Project-defined calls are resolved via the existing call graph $\text{CG}(e)$, while SDK calls are excluded since their bodies are unavailable. For function-type variables, potential targets are retrieved by combining type inspection and alias analysis, ensuring that variable-based indirect calls are correctly linked.

Second, the argument-level resolution complements this by capturing the actual function-type arguments that are executed at runtime. Instead of conservatively assuming all function-type parameters are invoked, our method determines the specific indices of parameters that are actually called inside each callee body. These indices are then mapped back to the caller's actual arguments to identify truly executed callbacks. For SDK functions, whose internal behavior is unknown, all function-type arguments are conservatively included to maintain soundness.

By combining expression-based and parameter-based resolution, our method produces a more accurate and semantically consistent callee set. Based on the enhanced call resolution, HAPFlow introduces more precise Call Edges and Exit-to-Return Edges when encountering call statements, significantly improves the precision of inter-procedural data-flow propagation within the IFDS solver. In the following section, we detail how these newly introduced edges are leveraged for taint propagation.

Closure. Closures capture external variables, extending their lifecycle across multiple scopes and increasing the complexity of data flow analysis. This issue becomes more prominent in ArkTS, where anonymous functions are widely used, making closure-related data flow problems more frequent.

Input: Call site c
Output: Set of resolved callees E
 $E \leftarrow \emptyset$;
 $e \leftarrow$ invoke expression of c ;
 $A_{func} \leftarrow \{ValueToArkMethod(a_i) \mid a_i.type = FunctionType\}$;
Step 1: Resolve from expression;
if e *is a project-defined function* **then**
 $E_{expr} \leftarrow CG(e) \setminus A_{func}$;
else
 if e *is a function-type variable* **then**
 $E_{expr} \leftarrow ValueToArkMethod(e)$;
 else
 $E_{expr} \leftarrow \emptyset$;
 end
end
 $E \leftarrow E \cup E_{expr}$;
Step 2: Resolve from arguments;
if e *is an SDK function* **then**
 $E_{param} \leftarrow A_{func}$;
else
 $I \leftarrow InvokedParams(E_{expr})$;
 $E_{param} \leftarrow \{A_i \mid A_i \in A_{func} \text{ and } i \in I\}$;
end
 $E \leftarrow E \cup E_{param}$;
return E ;

Algorithm 1: Enhanced Call Resolution in ArkAnalyzer

For example, in Listing 4, the `onCreate` function defines two variables, `info1` and `info2`, with `info1` being tainted. The function then calls `callA`, passing an anonymous function as an argument. Inside this anonymous function, both `info1` and `info2` are directly accessible, retaining the same values as in `onCreate`. Moreover, any modifications to these variables inside the anonymous function also affect their counterparts in `onCreate`. Therefore, the `sink(info1)` inside the anonymous function, as well as the `sink(info2)` in the `onCreate` function, will both lead to data leak. Additionally, `callA` defines a local variable named `info1`, assigning it a constant string before passing it to `sink` after executing `callback`. If taint propagation is handled incorrectly (e.g., directly propagating the taint from `onCreate`'s `info1` to `callA`'s local `info1`), false positives may arise.

Before performing taint propagation, the Intermediate Representation (IR) is transformed to make closure-captured variables explicit. For each function, all variables defined in an enclosing scope and used within the inner function are identified as captured variables. These captured variables are then packed into a synthetic `%closure` object, which serves as an additional argument in the function's call sites. In the callee, initialization statements are inserted before the first original statement to unpack each captured variable from `%closure` into a local variable within the function. This transformation of ArkIR ensures that closure-related data is explicitly represented and accessible, providing a clear interface for the subsequent closure-aware taint propagation.

```

1  function onCreate() {
2      let info1 = source();
3      let info2 = "";
4      callA(() => {
5          sink(info1); //leak
6          info2 = source();
7      });
8      sink(info2); //leak
9  }
10
11 function callA(callback: () => void) {
12     let info1 = "";
13     callback();
14     sink(info1); //no leak
15 }

```

Listing 4. Example demonstrating closure-based data leak

Algorithm 2 demonstrates how to integrate closure-aware taint propagation into an IFDS solver when the callee of the processing call site is an inner function of the caller. When the call site includes the %closure parameter, the algorithm iterates over each captured variable and checks if it corresponds to a current taint fact. For every match, the algorithm identifies the corresponding local variable within the callee and adds it to the set of facts to be propagated. Subsequently, these closure-related facts are paired with the callee’s first statement to form new IFDS facts. These new facts are then placed into the solver’s worklist for further propagation.

The above discussion focuses on closure-aware taint propagation from the caller to the callee. The handling of return flows is conceptually symmetric. When propagating facts along return edges, the analysis inspects the define statement of the tainted variable to determine whether it corresponds to a captured variable originally defined in the caller. If it does, the corresponding variable in the caller and the statement immediately following the call site are identified, and a new fact is generated and propagated to the caller’s control-flow graph. This symmetric treatment ensures that closure-related taints are tracked accurately across both call and return edges without modifying the core IFDS solver, preserving correctness and consistency in the lattice.

Through Algorithm 1 and Algorithm 2, HAPFlow effectively handles taint propagation scenarios involving closures. Figure 8 illustrates the extension of ArkIR and the propagation of closure variables within this process. Specifically, when a local variable in onCreate propagates its taint to the anonymous function, HAPFlow not only generates the usual Call Edge (connecting onCreate to callA) but also adds an extra Call Edge from onCreate to the anonymous function (represented by a solid yellow line and a dashed yellow line, respectively). Similarly, when the anonymous function completes execution, HAPFlow dynamically generates an ExitToReturn Edge (shown as a blue dashed line) that connects the anonymous function directly back to the statement following callA in onCreate, considering any changes made to closure variables. This approach ensures precise taint propagation for closure variables while preventing taint from erroneously flowing into unrelated functions, thereby reducing false positives and improving analysis accuracy.

Anonymous Function Param. As previously mentioned, the API design of OpenHarmony applications introduces special invocation patterns for certain source APIs. HAPFlow implements dedicated handling for such cases. As shown in Figure 9, when encountering these source APIs during analysis, HAPFlow will also add an additional Call Edge through Algorithm 1. In ArkIR, function parameters are initialized at the function entry point (e.g., account = %param1). Therefore, the newly added Call Edge connects the 0-point of the invocation statement to the callback function’s initialization statement %param1. The specific index of tainted parameters depends

Input: Caller *caller*, Callee *callee*, Data-flow fact *d*, IFDS worklist *W*
Output: Updated IFDS worklist *W'*
 $R \leftarrow \emptyset$;
 // Step 1: Check if call contains %closure argument
if *call arguments contain %closure* **then**
 %closure $\leftarrow$ the closure argument;
 // Step 2: Iterate over captured variables in closure
 foreach v_{caller} **in** %closure **do**
 if *d corresponds to* v_{caller} **then**
 $v_{callee} \leftarrow \text{getCapturedLocalInCallee}(v_{caller}, \text{callee})$;
 $R \leftarrow R \cup \{v_{callee}\}$;
 end
 end
end
 // Step 3: Propagate closure facts into IFDS solver
 $\text{firstStmt} \leftarrow$ first statement in *callee*;
foreach $v \in R$ **do**
 $\text{newFact} \leftarrow (v, \text{firstStmt})$;
 $\text{addToWorklist}(W, \text{newFact})$;
end
return *W*;

Algorithm 2: Closure Taint Propagation in IFDS Call Flow

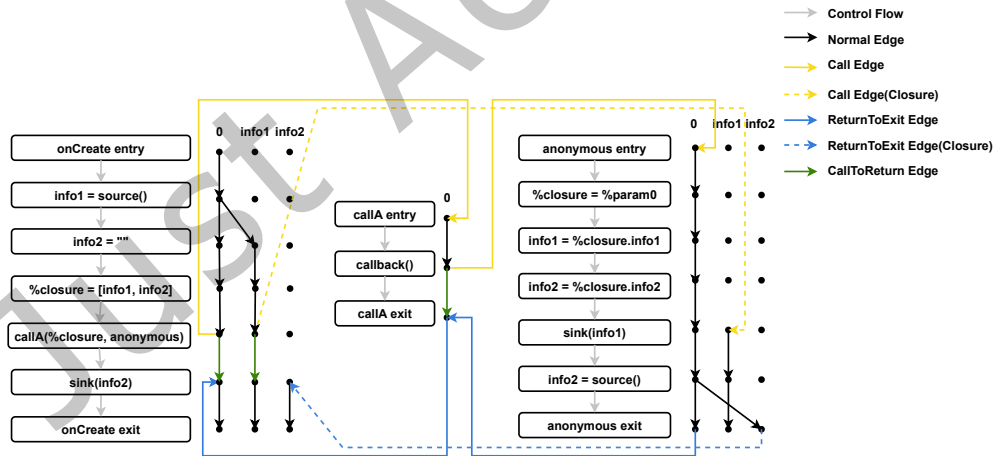


Fig. 8. Example of taint analysis within closures, Addressing the Challenge in Figure 1(b)

on the callback function. According to the results in Section 3.2, all callback-type sources that carry sensitive information implement either the Callback or AsyncCallback interface in the SDK definitions. In these definitions, Callback has a single parameter data, whereas AsyncCallback has two parameters: err and data. Therefore, the

tainted parameter index must be determined based on the specific type of the callback function. This mechanism ensures that taints can correctly propagate to the sensitive data produced by special APIs, thereby enhancing the coverage of taint analysis.

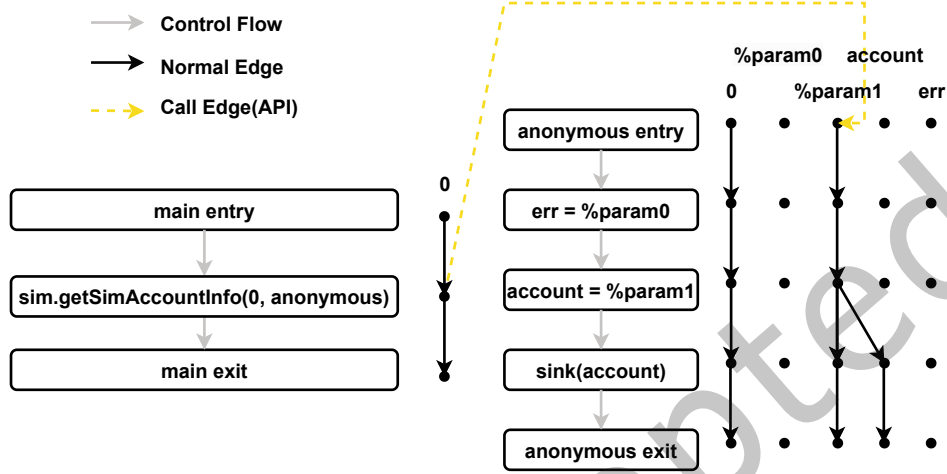


Fig. 9. Example of taint analysis for callback API, Addressing the Challenge in Figure 1(c)

The cross-function taint analysis approach proposed in this work, built upon an extended IFDS framework, also offers strong engineering extensibility. In particular, our modeling of closure-captured variables and function-as-value assignments does not rely on ArkTS-specific semantics but instead targets general programming patterns. These features likewise appear frequently in TypeScript, making the proposed mechanisms directly applicable to cross-function data-flow analysis tasks within the TypeScript ecosystem.

4 Experiment

Our experiment aims to address the following research questions:

RQ1: How frequently do identified source and sink APIs appear in real-world OpenHarmony applications?

RQ2: How accurate is HAPFlow in detecting taint flows in both benchmark test cases and real-world applications?

RQ3: What are the specific contributions and roles of each module within the overall analysis workflow of HAPFlow?

RQ4: How is HAPFlow's performance in terms of time consumption when analyzing real-world applications?

4.1 Experiment Setup

All experiments were conducted on a server with two Intel Xeon Gold 6330 CPUs, totaling 112 cores and operating at 2.00 GHz. The system is equipped with 503 GB of RAM. The software environment is based on Ubuntu 20.04 LTS, running on the Linux kernel version 5.15.0-126.

4.2 Dataset

To comprehensively evaluate the applicability of HAPFlow, we employed two distinct types of datasets for testing: (1) HapBench, a benchmark suite we specifically designed and constructed; and (2) a collection of real-world

OpenHarmony application projects gathered from open-source platforms. The following sections provide a detailed introduction of these two datasets.

4.2.1 The HapBench Benchmark. Although taint analysis tools and test suites exist for other languages and systems, such as web applications [45] and Android programs [44], there are currently no equivalent taint analysis tools for the OpenHarmony ecosystem. As a result, HAPFLOW lacks direct comparison benchmarks. To address this gap, we introduce HapBench, a dedicated benchmark for evaluating taint analysis tools in OpenHarmony applications.

We developed our benchmark by drawing upon the core principles and test case categories of DroidBench [9], a widely recognized taint analysis benchmark in the Android domain. On this foundation, we conducted a comprehensive extension and adaptation to accommodate the unique characteristics of the OpenHarmony platform. These extensions primarily focused on three key areas:

- (1) **Lifecycle Model Adaptations:** We designed specialized test cases that account for the unique multi-component and multi-lifecycle nature of OpenHarmony applications, including the distinct triggers for nested callback functions.
- (2) **ArkTS Language Feature Extensions:** To address the unique challenges of the ArkTS language, we specifically devised new test cases. These cases are tailored to capture data flows involving mutable variables within closures and implicit data flows that occur through state binding in the declarative UI framework.
- (3) **OpenHarmony-Specific APIs:** All test cases in HapBench exclusively utilize OpenHarmony-specific Source and Sink APIs. As previously described, OpenHarmony features different forms of Source APIs (e.g., return, argIn, callback), and we have systematically designed test cases for each of these distinct types.

Currently, HapBench encompasses a diverse set of taint propagation scenarios, ensuring a comprehensive assessment of HAPFLOW's capabilities. At present, HapBench consists of 7 categories with a total of 67 test cases, as detailed below:

- **Aliasing.** This category involves variable aliasing in programs, where multiple variables reference the same data object, complicating data flow analysis. The primary evaluation criterion is whether the analysis tool can accurately track taint propagation caused by aliasing and distinguish the actual influence range of different variables.

- **Anonymous Constructs.** Since anonymous structures can affect variable scope and data dependencies, this category tests whether the analysis tool can correctly parse taint propagation within anonymous functions and anonymous classes, including their access to and impact on external variables.

- **Array-like Structures.** This category focuses on taint propagation in data structures such as arrays, sets, and maps. These data structures support various access patterns, including index-based and key-value lookups, which influence data flow. The key evaluation criterion is whether the analysis tool can correctly handle taint flow in different data structures and whether it possesses field sensitivity.

- **Field and Object Sensitivity.** This category evaluates the analysis tool's sensitivity at both the object and field levels. Different fields within an object or different instances of a class may exhibit independent data flows. The tool must accurately distinguish between these fields or instances to prevent erroneous taint propagation from one field or instance to an unrelated one.

- **General Language Features.** This category covers common ArkTS programming constructs, including closures, parameter passing, exception handling, inheritance, and polymorphism. The objective is to assess whether the analysis tool can effectively track taint propagation within these fundamental language features.

- **Lifecycle Modeling.** This category examines lifecycle functions and callback functions. Since these functions are not explicitly invoked in the code but are instead triggered by the system framework, the evaluation focuses

on whether the analysis tool can correctly model their execution process and identify potential taint propagation paths.

- **OpenHarmony Specific APIs.** This category covers OpenHarmony-specific APIs, which may involve external data inputs and serve as potential taint sources. The evaluation focuses on whether the analysis tool can accurately identify taint propagation paths related to OpenHarmony APIs and effectively model their data interactions.

4.2.2 Real Applications. Our dataset consists of 3,490 applications, with all data sourced from open-source code hosting platforms. The structure of the data sources is clear: 2,516 applications originated from official organizations on Gitee³: the **OpenHarmony**, **OpenHarmony-SIG** (Special Interest Groups), and **OpenHarmony-TPC** (Third-Party Components), which mainly comprise system tools and official example applications. The remaining 974 applications were obtained from search pages on GitHub and Gitee (excluding the official organization section above), predominantly consisting of various community-contributed demos or small functional applications. Together, this dataset represents both standard and community-driven scenarios in the OpenHarmony ecosystem, supporting the relevance and generalizability of our evaluation results.

4.3 RQ1: How frequently do identified source and sink APIs appear in real-world OpenHarmony applications?

To investigate this research question, we conducted a large-scale empirical study on real-world OpenHarmony applications collected in our dataset. Specifically, we developed an automated script to scan the codebases of these projects and record the actual usage of the previously identified source and sink APIs. This process enabled us to detect whether these APIs appear in practice, as well as to quantify their usage frequency across different projects.

The analysis revealed that the majority of the identified APIs are indeed used in practice. Out of the 2066 source APIs and 172 sink APIs, 1173 sources and 87 sinks were actively invoked in real-world applications. This finding suggests that sensitive data flows are not merely theoretical but are highly likely to occur in practice, underscoring the potential risk of data leak in the OpenHarmony ecosystem and highlighting the necessity of taint analysis tools.

We further examined API usage from two perspectives: (1) the number of projects in which each API appeared, and (2) the total call counts aggregated across all projects. The results show that some APIs are not only widely adopted but also invoked very frequently, making them particularly critical from a security perspective. For sink APIs, the most frequently used ones include logging APIs provided by the `hilog` module, as well as APIs for posting message through the `worker.ThreadWorker.postMessage`. For source APIs, those related to program object information (e.g., `abilityDelegatorRegistry` module) component communication information (e.g., `app.ability.UIAbility`), and subscription to public events (e.g., `@ohos.commonEventManager` and `@ohos.commonEvent`) were among the most widely used.

Overall, these results indicate that OpenHarmony applications frequently rely on both sensitive sources and sinks in their everyday functionality. This prevalence, combined with the potential for source-to-sink flows, reinforces the importance of developing specialized taint analysis techniques for the OpenHarmony environment. Detailed usage statistics are provided in Table 2 and Table 3, where only APIs with more than 100 total call instances are reported. It is important to note that the experimental results exclusively feature OpenHarmony-specific APIs. Although general APIs such as `console.log` were also analyzed and treated as potential sinks in our analysis, they are omitted from the table as they are not unique to the OpenHarmony framework.

³a major Chinese code hosting platform similar to GitHub, <https://gitee.com/>

Table 2. The usage of the sink API

API	App Count	Sum
hilog.info	2923	70199
hilog.error	2680	6433
hilog.debug	578	909
worker.ThreadWorker.postMessage	47	786
hilog.warn	568	759
http.HttpRequest.request	105	627
fs.writeSync	219	623
rpc.RemoteObject.sendMessageRequest	13	255
rpc.RemoteObject.sendRequest	24	246
relationalStore.RdbStore.update	30	136

Table 3. The usage of the source API

API	App Count	Sum
abilityDelegatorRegistry.getAbilityDelegator	2323	6112
abilityDelegatorRegistry.getArguments	2706	5412
app.ability.UIAbility.onCreate	2960	5266
resourceManager.ResourceManager.getStringValue	148	1587
commonEventManager.subscribe	99	783
AbilityDelegator.getAppContext	437	765
relationalStore.ResultSet.getString	37	613
sensor.getSingleSensor	3	545
commonEvent.subscribe	85	531
Context.getApplicationContext	278	521

Answer to RQ1

These findings indicate that source and sink APIs are frequently used in real OpenHarmony projects, revealing a widespread potential risk of data leak, and presenting the effectiveness of our identification of source and sink APIs for OpenHarmony applications. This underscores the urgent need for systematic detection and protection through taint analysis tools.

4.4 RQ2: How accurate is HAPFLOW in detecting taint flows in both benchmark test cases and real-world applications?

4.4.1 Performance on the HapBench Benchmark. To rigorously evaluate the accuracy and reliability of HAPFLOW, we conducted a series of comprehensive experiments on HapBench, a benchmark specifically designed for OpenHarmony applications. HapBench consists of seven categories of test cases, each designed to thoroughly examine challenges that static analysis tools typically face. Our evaluation primarily focused on two key metrics: Precision and Recall. Precision measures the proportion of reported taint flows that are indeed valid, while Recall measures the proportion of all actual taint flows that are correctly identified by our tool.

```

1 let a = {
2   info: JSON.stringify(geoLocationManager.getLastLocation()), //source
3 }
```

Table 4. Test Results of HAPFlow on HapBench

Category	Count	TP	TN	FP	FN
Aliasing	6	5	1	0	0
Anonymous Constructs	10	7	2	0	1
Array-Like Structures	5	4	0	1	0
Field And Object Sensitivity	6	2	4	0	0
General Language Features	22	17	4	0	1
OpenHarmony Specific APIs	5	3	1	0	1
Lifecycle Modeling	13	12	0	1	0
Total	67	50	12	2	3
Precision = 96.15%		Recall = 94.34%			

```

4   let b = {
5       n: zero(a),
6   }
7   }
8
9   function zero(a: {info}): number {
10      hilog.info(0x0000, 'testTag', '%{public}s', a.info); //sink, leak
11      return 0;
12  }

```

Listing 5. Data leak in AnonymousClass2 of HapBench

As shown in Table 4, HAPFlow achieved an overall precision of 96.15% and recall of 94.34%. These results reflect three key contributors to high precision. First, HarmonyMain provides a complete and explicit abstraction of Declarative UI lifecycle semantics, covering all lifecycle- and callback-triggered execution sequences. Second, the IFDS framework ensures flow- and context-sensitive propagation based on a sound theoretical foundation, preventing unnecessary taint expansion. Third, by encoding closure semantics directly into ArkIR and applying a dedicated propagation algorithm, HAPFlow effectively handles captured variables within the evaluated test cases. Our experimental results on HapBench suggest that these design decisions enable HAPFlow to deliver precise and semantically aligned analysis results for various common closure patterns in OpenHarmony applications. Together, these design decisions enable HAPFlow to deliver precise and semantically aligned analysis results across diverse OpenHarmony scenarios.

While HAPFlow is highly accurate, it entails inherent approximations as a static analysis framework. Specifically, HAPFlow may introduce over-approximation when resolving complex lifecycle relationships or handling aggregate data structures such as arrays. Conversely, under-approximation can occur when certain dynamic call patterns or implicit framework behaviors are not fully captured by the static call graph. These theoretical properties manifest as specific limitations in practice. In the following, we provide a closer inspection of our results, analyzing how these approximations lead to false positives or negatives in specific categories.

Array-Like Structures: In this category, we observed one false positive. This limitation stems from the domain-insensitive nature of our current implementation. When an element within an array or array-like structure is tainted, HAPFlow conservatively treats the entire aggregate object as tainted. As a result, even when a sink operation involves only untainted elements of the array, a taint flow may still be reported. Addressing this issue requires the integration of more array-like field-sensitive analysis in future work.

OpenHarmony Specific APIs: This category exhibited one false negative. HAPFLOW is currently unable to determine whether different file paths refer to the same file, and therefore fails to detect data leaks arising from cross-file operations. This limitation highlights a deficiency in cross-file alias analysis. Overcoming this challenge will require a more sophisticated global alias analysis to precisely track and map complex cross-file object references.

General Language Features and Anonymous Constructs: Across these two categories, we identified three false negatives. Detailed examination traced these errors back to limitations of our underlying static analysis engine, ArkAnalyzer, particularly during the language-to-intermediate-representation (IR) translation phase. Specifically, logical errors occurred when transforming certain ArkTS constructs (e.g., try-catch statements) into three-address code, disrupting taint propagation paths and causing missed flows. Moreover, when handling complex anonymous functions or nested inner functions, ArkAnalyzer occasionally produced incomplete information. For example, within the **Anonymous Constructs** category, a test case named `AnonymousClass2` was designed to evaluate taint propagation through nested inner functions. The relevant code is shown in Listing 5. This scenario involves two anonymous classes: variable `a` is an instance of an anonymous class, and its property `info` is tainted as a taint source. Variable `b` is also an instance of an anonymous class, where one of its properties is initialized with a function. Within this function, an output operation is performed on `a.info`, leading to a sensitive data leak. Ideally, the initial values of classes should appear in ArkAnalyzer as the Intermediate Representation (IR) of the constructor. However, in some complex context of anonymous classes, the IR information for this part is incomplete or missing, which ultimately causes a false negative.

Lifecycle Modeling: Although HarmonyMain significantly improves path coverage by explicitly modeling declarative UI and component lifecycle behavior, certain sources of false positives remain. HarmonyMain adopts a conservative abstraction in which lifecycle methods and callback functions may be triggered in arbitrary orders. This may introduce execution paths that are unlikely to occur in real applications. For example, if a field of a `UIAbility` is marked as a Source in `onDestroy` and later flows into a Sink in `onCreate`, the analysis may report a false positive. This corresponds to a representative false-positive scenario within this category. However, when tainted data is written into static fields, singleton objects, application-level global state, or other cross-component shared objects, the corresponding data-flow path may indeed exist after the `UIAbility` is destroyed and subsequently recreated. Therefore, such flows are feasible under certain conditions and are not necessarily false positives.

We plan to address these identified limitations in future work. By incorporating field-sensitive analysis of Array, implementing a robust global alias analysis, and refining translation process of IR, we aim to further enhance the accuracy and soundness of HAPFLOW.

4.4.2 Performance on Real Applications. In our evaluation on over 3,000 open-source OpenHarmony applications, we focus our analysis on data flows originating from high-sensitivity source APIs. This choice aligns with practical privacy risks, as high-sensitivity sources (e.g., system identifiers, account data, user information) are most likely to lead to security or compliance issues when leaked. Medium- and low-sensitivity sources may still cause information exposure, but their impact is highly context-dependent. Therefore, we report flow-analysis and validation results with respect to high-sensitivity sources, which more accurately reflect the security significance of potential leaks. Ultimately, we identified 73 potential data leak flows.

Table 5 presents the top ten most frequent source-sink pairs by occurrence count. We observe that the sinks involved in most high-frequency data flows are logging functions (such as `console.log` or `hi.log.info`). The identified sources include sensitive data like geolocation, cookies, audio streams, and connected device information. Overall, this distribution indicates that while severe external leakage (e.g., network transmission) is infrequent in open-source applications, the writing of privacy-related data to log output remains a primary and recurring pattern. This finding underscores the critical need for automated detection prior to application deployment.

Table 5. leak data-flow in open-source Apps

Source	Sink	Num
webView.GeolocationPermissions.getAccessibleGeolocation	console.log	8
webView.WebCookieManager.getCookie	console.log	7
bluetooth.getPairedDevices	console.log	6
webView.WebCookieManager.getCookie	hilog.info	4
audio.AudioCapturer.read	fs.write	4
connection.getPairedDevices	console.log	4
webView.WebCookieManager.fetchCookie	console.log	3
connection.getPairedDevices	console.log	3
sms.createMessage	hilog.info	3
sms.getDefaultSmsSlotId	hilog.info	3

We now select one instance from the detection results for case analysis.

```

1  @Entry
2  @Component
3  struct Index {
4      aboutToAppear(): void {
5          this.getPairDevice();
6      }
7
8      print(msg: string) {
9          ...
10         console.log(msg);
11     }
12
13     getPairDevice(){
14         let macs: Array<string> = bluetooth.getPairedDevices();
15         ...
16         this.print(JSON.stringify(macs));
17     }
18 }

```

Listing 6. Data leak in BluetoothSocketTest

Listing 6 illustrates a potential data leak example (showing only the relevant source and sink parts) from a project. In the application, the Index UI component declares an aboutToAppear lifecycle function, which executes just before the component becomes visible. This function calls getPairDevice, which uses the system API bluetooth.getPairedDevices to retrieve Bluetooth paired device information. This data is then printed using an internally wrapped logging function. This logging practice may lead to the disclosure of sensitive data (paired device information), posing a potential data leakage risk.

Among the 73 detected data leakage results, eight were identified as false positives. Six of them were caused by taint propagation for SDK function calls. Our analysis marks the left variable of an assignment as tainted whenever the right side is a SDK function call containing tainted arguments. While this approximation is reasonable for transformation-oriented APIs such as JSON.stringify, which commonly preserve the original information, it becomes inaccurate when the invoked SDK function does not return data derived from the tainted input.

For instance, one case involved passing a tainted variable to `fs.write`, whose return value `writtenSize` only represents the length of written data. Since `writtenSize` was later logged to the console, it was incorrectly reported as a leakage.

The remaining two false positives were due to our field-insensitive treatment of aggregated data structures. When an element of an array or map is tainted, the entire structure is conservatively marked as tainted. Consequently, subsequent operations retrieving different elements were still flagged as leaks, leading to incorrect results.

For the large-scale analysis on open-source OpenHarmony applications, all reported source–sink paths were manually inspected to determine whether each data flow was semantically feasible. Although we attempted to contact project maintainers to further validate these findings, most repositories were inactive and no responses were received.

To complement the community evaluation, we analyzed several large commercial OpenHarmony applications provided by industrial partners. Among the detected flows, five privacy-related cases were confirmed by the developers. Table 6 summarizes the five detected leaks in these applications involving the propagation of sensitivity account or system information into logging APIs. These confirmations demonstrate that the tool is able to identify practically relevant taint-propagation issues in production environments.

Table 6. confirmed leak data-flow in real Apps

Source	Source Sensitivity	Sink
<code>accountManager.getOsAccountLocalIdForUid</code>	high	<code>hilog.info</code>
<code>accountManager.getOsAccountLocalIdForUid</code>	high	<code>hilog.warn</code>
<code>accountManager.getOsAccountLocalId</code>	high	<code>hilog.info</code>
<code>settings.getValueSync</code>	medium	<code>hilog.info</code>
<code>commonEventManager.subscribe</code>	medium	<code>hilog.error</code>

Answer to RQ2:

HAPFLOW demonstrates high precision and recall across diverse scenario tests in HapBench, while also effectively identifying real potential sensitive data leaks in real-world applications with a low false-positive rate. Overall, it maintains a consistently high level of detection accuracy.

4.5 RQ3: What are the specific contributions and roles of each module within the overall analysis workflow of HAPFLOW?

To evaluate the effectiveness of the novel technical components introduced in HAPFLOW, we conducted an ablation study. Specifically, we systematically disabled each key strategy and then used HapBench and the real-world application dataset for testing, comparing the results against the baseline established in RQ2. The three new experimental groups are as follows:

hf1: Disables the connection and modeling of potential entry functions in OpenHarmony applications; instead, it analyzes solely from the `onCreate` function of the `EntryAbility` class (the first lifecycle function of the default initial component) as the entry point.

hf2: Disables the taint propagation strategy for closure variables, meaning tainted variables from outer functions no longer affect inner functions.

hf3: Disables the source recognition mechanism specifically designed for APIs that pass anonymous functions as parameters.

Table 7. ablation experiment results on HapBench

Category	Count	hf1				hf2				hf3			
		TP	TN	FP	FN	TP	TN	FP	FN	TP	TN	FP	FN
Aliasing	6	5	1	0	0	5	1	0	0	5	1	0	0
Anonymous Constructs	10	7	2	0	1	3	2	0	5	7	2	0	1
Array-Like Structures	5	4	0	1	0	4	0	1	0	4	0	1	0
Field And Object Sensitivity	6	2	4	0	0	2	4	0	0	2	4	0	0
General Language Features	22	13	4	0	5	14	4	0	4	17	4	0	1
OpenHarmony Specific APIs	5	2	1	0	2	3	1	0	1	2	1	0	2
LifeCycle Modeling	13	0	1	0	12	12	0	1	0	12	0	1	0
Total	67	33	13	1	20	44	12	2	9	49	12	2	4
Precision		97.05%				95.65%				96.08%			
Recall		62.26%				83.02%				92.45%			

The result of the ablation experiments on the HapBench benchmark is shown in Table 7, the differences between the three experimental groups and HAPFLOW were primarily reflected in false negatives. This is because each experimental group partially weakened or removed HAPFLOW’s ability to detect data leak patterns unique to OpenHarmony. Specifically:

- hf1 showed the largest increase in false negatives. In particular, 12 test cases in the LifeCycle category resulted in missed detections. Additional issues also appeared in the GeneralArkTS category. For example, in the StaticFieldInit case, a static property of a user-defined class was initialized with tainted data. Without the modeling support of HarmonyMain, such data flows could not be captured, leading to false negatives.
- hf2 exhibited false negatives primarily in the Anonymous category. Since support for closure semantics was removed, many data leak scenarios involving anonymous functions went undetected. Moreover, within the GeneralArkTS category, several test cases relying on closure features also produced errors.
- hf3 introduced relatively few additional false negatives, with only one observed. This is because although callback-type source APIs are diverse in number, their usage patterns are highly similar. Consequently, HapBench included only a single test case of this kind in the HarmonySpecific category, leading to minimal differences in the benchmark evaluation.

In the real-world application tests, hf1, hf2, and hf3 detected 4, 73, and 47 potential data leak issues, respectively, compared to the 73 issues originally identified by HAPFLOW. All three groups showed a clear reduction in detection capability, with hf1 experiencing the sharpest decline. This further underscores the critical role of HarmonyMain as a virtual entry function in enabling comprehensive analysis. The results for the hf2 group showed no deviation from the original findings. This suggests that while the closure handling is theoretically vital for soundness, the specific open-source applications in our test set do not contain complex, exploitable data flows critically dependent on closure variables. Nevertheless, retaining this mechanism is essential for ensuring theoretical completeness and soundness against potential, yet rare, closure-based vulnerabilities. In addition, although hf3 showed little difference from HAPFLOW on HapBench, it detected fewer issues than hf2 in real-world applications. This suggests that while callback-type source APIs have limited test coverage in HapBench, they are more prevalent in practice and thus pose greater potential risks.

Answer to RQ3

Each module in HAPFlow plays an indispensable role in the overall analysis pipeline. Removing any single component would inevitably compromise the effectiveness of the system, resulting in a noticeable decline in its detection capability and accuracy.

4.6 RQ4: How is HAPFlow's performance in terms of time consumption when analyzing real-world applications?

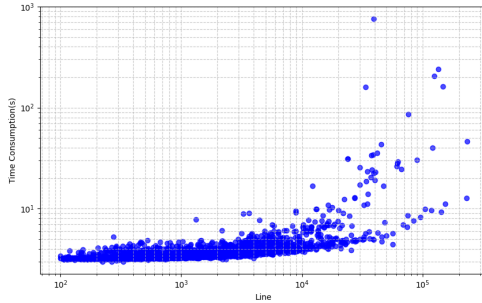


Fig. 10. Tool Runtime Analysis on Real-World Applications

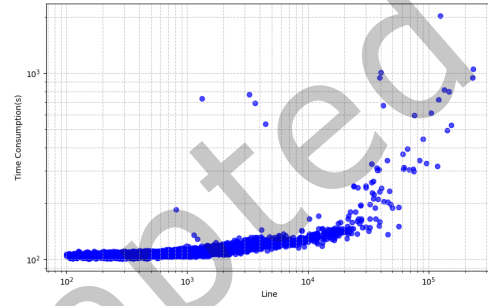


Fig. 11. Tool Memory Consumption Analysis on Real-World Applications

Figure 10 illustrates the time consumption of HAPFlow when analyzing 3,490 open-source OpenHarmony projects, with a comparison to project code size. The time shown in the figure represents the total duration for each project starting from constructing the repository scene structure, including both alias analysis and taint analysis. Overall, there is a clear positive correlation between project size and analysis time. For the vast majority of projects with fewer than 10,000 lines of code, HAPFlow completes the analysis within 10 seconds, demonstrating high efficiency. As project size exceeds 10,000 lines, the required analysis time increases significantly, with the maximum observed time approaching over 700 seconds. Further statistics indicate that among all 3,490 projects, only 40 projects (approximately 1.15%) required more than 10 seconds, and only 5 project (approximately 0.14%) exceeded 100 seconds. This shows that, except for a few extremely large projects, HAPFlow maintains fast analysis speeds in most real-world scenarios. To provide a clearer picture of our framework's core efficiency, we separately measured the preprocessing time (via ArkAnalyzer) and the core taint analysis time (via HAPFlow). Our evaluation reveals that the core taint analysis is highly efficient, accounting for an average of 9.53% of the total execution time. This overhead varies with the scale of the project: for small-scale projects (<10,000 lines), the analysis time share is only 7.20% on average, whereas for large-scale projects ($\geq 10,000$ lines), it increases significantly to 37.44%, reflecting the higher complexity of data-flow propagation in larger applications.

Figure 11 shows the memory consumption of HAPFlow. Most projects have an RSS memory usage of around 100 MB, indicating overall low memory demand. As the scale of analysis grows, memory consumption increases, reaching a maximum of 1 GB. Specifically, 49 projects (approximately 1.40%) consumed more than 200 MB, and only 3 project (approximately 0.09%) exceeded 1 GB. These results indicate that HAPFlow remains lightweight in terms of memory usage in most cases, with higher consumption only occurring in a very small number of extremely large projects.

Beyond the general growth trend with project size, our analysis indicates that the analysis time is correlated with the number of source APIs used in codes. The most significant gap is observed in scenarios with code sizes

ranging from 30,000 to 40,000 lines, we observed that a project with 7 sources required 11.15 seconds, while another project with 218 sources required 758.25 seconds. Notably, while the number of source APIs and lines are intuitive indicators of performance, they are not the sole determinants. Ultimately, the analysis performance depends on the breadth and depth of taint fact propagation within the application's logic.

To further understand the performance characteristics, it is important to note that both ArkAnalyzer and HAPFLOW are designed with a strong emphasis on time efficiency. During implementation, several engineering decisions were intentionally made to accelerate analysis of large-scale OpenHarmony projects. Specifically, the frameworks maintain multiple layers of caching structures to avoid repeated computation of fundamental program facts, including control-flow graphs for each statement and block, call-site to callee mappings and resolved call relationships, variable def-use information within functions, and so on.

However, this design choice also leads to higher memory consumption, especially for very large projects. Cached control-flow information, preprocessed call relations, and def-use graphs all occupy memory proportional to project size. This explains the distribution shown in Figure 11. The elevated memory footprint is a deliberate trade-off made to ensure fast and scalable analysis in practice.

Answer To RQ4

Overall, HAPFLOW demonstrates good scalability and practicality in both time and memory consumption, maintaining reasonable resource usage across projects of varying sizes, and thus meeting the requirements of large-scale static analysis in real-world applications.

5 Threats to Validity

5.1 Internal threats

There are two main internal threats. The first lies in the construction of the HarmonyMain function. To ensure sufficient analysis coverage, HAPFLOW simulates all possible combinations of lifecycle function invocation orders when building the analysis entry point. This approach may introduce execution sequences that do not occur in real applications (e.g., invoking `onDestroy` before `onCreate`), potentially leading to false positives. However, such cases are extremely rare in practice, and no false positives caused by this modeling assumption were observed in our real-world application experiments.

The second threat concerns asynchronous execution. Currently, HAPFLOW assumes that all functions execute synchronously and does not explicitly model asynchronous task scheduling or callback execution orders. As a result, data flows that rely on asynchronous context switches may be missed. We plan to support this aspect in future work.

5.2 External threats

The first external threat is that the final confirmation of source-sink definitions and the validation of real applications' data-flow results rely on manual inspection. Although multiple reviewers were involved and cross-checked the outcomes, human judgment is inherently subjective and may introduce inaccuracies. Therefore, the generalizability of the identified sensitive flows could be affected by potential human errors in the labeling and verification process.

The second external threat is the limited maturity of third-party libraries in the OpenHarmony ecosystem. Unlike Android, where many privacy-sensitive behaviors originate from external dependencies, OpenHarmony currently offers very few widely adopted third-party packages. Most real-world applications rely almost exclusively on system APIs. As a result, our source-sink specification does not include third-party libraries. Extending our modeling to such libraries will be an important direction for future work.

Another external threat is the completeness of source/sink APIs. Despite our LLM-assisted identification of 10,000+ APIs, absolute coverage is unattainable due to rapid SDK evolution. Any overlooked APIs may lead to under-approximation (false negatives). To mitigate this, HAPFlow provides an extensible configuration mechanism for users to update the API set manually, and we will continue to incorporate newly identified APIs in our future work.

6 Related Works

Taint analysis has been widely studied as a key technique in program security, especially for vulnerability detection, privacy protection, and software optimization. Static taint analysis infers data flows without executing the program, typically by constructing control flow graphs (CFGs) and data flow graphs (DFGs). Numerous static analysis frameworks have been developed for various programming languages, including Soot, Tai-e and Doop for Java [11, 17, 37, 39], SVF and PhASAR for C/C++ [34, 36], ESLint and TAJs for JavaScript [2, 16], and RUPTA for Rust [21]. Building on these frameworks, advanced static taint analysis tools have been proposed.

For instance, FlowDroid [9] leverages the IFDS algorithm to perform context-sensitive and flow-sensitive taint analysis for Android applications, and introduces on-demand alias analysis to improve precision. IccTA [19] extends FlowDroid by handling inter-component communication (ICC) in Android apps, significantly improving the detection of privacy leaks across components. Other notable tools, such as SCAnDroid [35], CHEX [26], DroidSafe [14], and Amandroid [43], further advance Android-specific taint analysis by targeting issues like side-channel leaks and component hijacking.

In the context of large-scale enterprise systems, ANTaint [42] and CFTaint [46] demonstrate scalability and precision improvements. Additionally, declarative languages such as Datalog have been employed for taint analysis. Early approaches, such as the work by Livshits [25], faced limitations due to the lack of pointer analysis. P/Taint [15] later integrated pointer analysis into a Datalog-based framework to enhance precision, though at the cost of increased analysis time for large-scale systems.

Building upon general taint-analysis frameworks, a related line of work focuses on the identification of security-sensitive APIs, typically formalized as source and sink definitions that guide the analysis. SuSi [8] pioneered the field by using machine learning on Android source code features to automatically classify and grade sensitive APIs. CODOC [33] attempted to improve this by integrating source code and documentation with deep learning, but confirmed the inherent difficulty of the task due to persistent high false-positive rates in real-world application. DocFlow [38] provided an alternative, documentation-driven approach, using NLP to extract taint specifications, making it effective for analyzing closed-source libraries. Bluebird [40] utilized probabilistic inference to analyze security checks in pre-installed applications, thereby inferring and auditing the proper access control specifications for proprietary APIs in customized Android systems.

However, these methods cannot be directly applied to OpenHarmony, as its system architecture and runtime environment fundamentally differ from Android, utilizing a distinct programming language ecosystem (e.g., C/C++, JS, ArkTS). Therefore, we have specifically tailored prompts and examples to address the unique technical characteristics of the OpenHarmony platform.

To evaluate the effectiveness and precision of Android taint analysis tools, the research community has proposed multiple benchmark suites. DroidBench [9] is a micro-benchmark proposed to test the accuracy of tools in handling various Java and Android language features. ICCBench [19] serves as a crucial complement, focusing specifically on complex taint leak issues arising from Inter-Component Communication. UBCBench [44] is a small, targeted benchmark that focuses on assessing a tool's ability to handle data flow within specific Android features, such as SharedPreferences. To evaluate the effectiveness and scalability of ANTaint in industrial SOA applications, Wang et al. [42] developed a new micro-benchmark suite, SOFABENCH. ECSTATIC [29] creates high-quality, realistic, and easily scalable static analysis benchmarks by automatically discovering and extracting

specific analysis feature patterns from real programs. To enhance the realism and coverage of these benchmarks, researchers have also introduced more comprehensive suites: Ghera [28] is a broader Android vulnerability benchmark library that not only includes taint flow but also covers more complex security flaws.

While existing test case suites offer general applicability across common scenarios, including control flow analysis, virtual dispatch in inheritance, field sensitivity, and alias analysis. They significantly lack coverage for OpenHarmony and ArkTS-specific features. Our constructed HapBench directly addresses and mitigates this gap.

These gaps highlight the unique analysis challenges introduced by ArkTS and ArkUI. Recent efforts for the OpenHarmony ecosystem focus on building general static analysis infrastructures. In particular, ArkAnalyzer [12] provides program abstractions, ArkIR, control flow graph, call graph, and so on. While ArkAnalyzer lays the essential framework for code analysis, it does not implement precise inter-procedural taint propagation—motivating the need for HAPFLOW.

Beyond static analysis, HapTest[23] introduces a dynamic testing framework specifically designed for OpenHarmony applications, while HapRepair [22] leverages RAG techniques to automate the repair of defects in OpenHarmony apps. Although these studies demonstrate the community’s growing attention to the OpenHarmony ecosystem, they either focus on testing and repair or provide only foundational analysis infrastructure, and thus lack a dedicated taint-analysis capability tailored to ArkTS.

7 Conclusion

In this paper, we propose HAPFLOW, the first taint analysis framework specifically designed for native OpenHarmony applications. Built on top of the static analysis framework ArkAnalyzer, HAPFLOW fully incorporates the characteristics of OpenHarmony applications, including lifecycle functions and event callback functions. Furthermore, we enhanced the IFDS algorithm by adapting it to the syntactic features of ArkTS, thereby achieving precise inter-procedural data flow analysis.

Beyond the ArkTS platform, the techniques developed within HAPFLOW offer significant generalizability. Our lifecycle modeling approach provides a transferable analysis model for modern declarative UI applications, such as Android’s Jetpack Compose. Furthermore, our sophisticated cross-function data flow analysis component can be readily extended to the TypeScript language.

We extracted 2066 source APIs and 172 sink APIs through a combination of large language model analysis and manual verification, and used this set as the configuration for HAPFLOW to conduct comprehensive experiments. On the benchmark suite HapBench, HAPFLOW achieved a precision of 96.15% and a recall of 94.34%. When applied to over 3,000 real-world open-source OpenHarmony projects, HAPFLOW successfully identified 73 actual data-flow paths with 8 false positives, demonstrating its high accuracy. We further performed ablation experiments on HAPFLOW to validate the effectiveness and contribution of each individual module. In terms of performance, HAPFLOW was able to complete the analysis for the majority of projects in 10 seconds and 200 MB.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62572024), by the National Key Research and Development Program of China (No. 2024YFB4506300), by “the Fundamental Research Funds for the Central Universities”.

A Appendix

Table 8. Prompt Template for Classifying APIs as Source, Sink, or None

Task Objective	Determine whether the given function belongs to a <i>Source</i> , <i>Sink</i> , or <i>None</i> category based on its behavior regarding sensitive data.
Source	A function is considered a Source if it provides sensitive data directly or indirectly. There are three types of Sources: • returnSource: The return value itself contains sensitive data. • paramSource: One or more input parameters are tainted (contain sensitive data). • callbackSource: A callback function parameter receives sensitive data through its callback argument. If the API is a Source: • For paramSource type: output the corresponding parameter index (starting from 1). • For callbackSource type: output the parameter index (starting from 1) of the callback function that receives tainted data.
Sink	A function is considered a Sink if it explicitly transmits parameter data outside the application boundary, such as via network, SMS, system I/O, external storage, log output, or clipboard operations.
None	A function that neither accesses sensitive data nor transmits data outside the application boundary. It performs only normal logic or UI operations.
Input Information	The following fields are provided for each API: • api_name: API name • class_name: class to which it belongs • namespace: namespace (if any) • file: file path • parameters: parameter list • return_type: return type • description: documentation or comments
Output Format	Use the following keys (in English, unified format): • api_name • api_type: source / sink / none • source_type: returnSource / paramSource / callbackSource / null • tainted_param_index: if paramSource or callbackSource, output the parameter index (integer); otherwise, output -1 • reason: English explanation describing the rationale
Examples	1. @ohos.geoLocationManager.getLastLocation(): Location api_type: source source_type: returnSource tainted_param_index: -1 reason: The function returns the user's location, which is sensitive data.

Continued on next page

Table 8 – continued from previous page

Task Description	(continued)
	2. <code>@ohos.app.ability.UIAbility.onCreate(want: Want, launchParam: AbilityConstant.LaunchParam)</code> <code>api_type</code>: source <code>source_type</code>: paramSource <code>tainted_param_index</code>: 1 reason: The parameter <code>want</code> may contain device communication information, which is sensitive. 3. <code>@ohos.pasteboard.SystemPasteboard.getData(callback: AsyncCallback<PasteData>)</code> <code>api_type</code>: source <code>source_type</code>: callbackSource <code>tainted_param_index</code>: 1 reason: The callback function parameter returns clipboard data, which involves user privacy. 4. <code>@ohos.telephony.sms.sendShortMessage(options: SendMessageOptions, callback: AsyncCallback<void>)</code> <code>api_type</code>: sink <code>source_type</code>: null <code>tainted_param_index</code>: -1 reason: Sending an SMS transmits data outside the app boundary, which is a sensitive operation. 5. <code>@internal.full.globalsetTimeout(handler: Function string, delay?: number, ...arguments: any[])</code> <code>api_type</code>: none <code>source_type</code>: null <code>tainted_param_index</code>: -1 reason: This function only delays code execution and does not involve sensitive data or operations.

References

- [1] [n. d.]. CodeQL. <https://codeql.github.com/> Accessed: November 1, 2025.
- [2] [n. d.]. Find and fix problems in your JavaScript code. <https://eslint.org/> Accessed: July 9, 2024.
- [3] [n. d.]. jetpackcompose. <https://jetpackcompose.cn/docs/tutorial/> Accessed: November 1, 2025.
- [4] [n. d.]. privacy protection. <https://docs.openharmony.cn/pages/v4.1/zh-cn/device-dev/security/security-privacy-protection.md> Accessed: July 9, 2025.
- [5] [n. d.]. pysa. <https://github.com/facebook/pyre-check> Accessed: November 1, 2025.
- [6] 2024. Introduction to ArkTS: The Native Development Language of HarmonyOS. <https://developer.huawei.com/consumer/cn/blog/topic/03176639732307027>. Accessed: 2025.3.19.
- [7] 2025. ArkTS. <https://developer.huawei.com/consumer/cn/arkts/>. Accessed: 2025.3.19.
- [8] Steven Arzt, Siegfried Rasthofer, and Eric Bodden. 2013. Susi: A tool for the fully automated classification and categorization of android sources and sinks. (2013).
- [9] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *ACM sigplan notices* 49, 6 (2014), 259–269.
- [10] Dan Boxler and Kristen R Walcott. 2018. Static taint analysis tools to detect information flows. In *Proceedings of the International Conference on Software Engineering Research and Practice (SERP)*. The Steering Committee of The World Congress in Computer Science,

- Computer ..., 46–52.
- [11] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*. 243–262.
 - [12] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. *arXiv preprint arXiv:2501.05798* (2025).
 - [13] Yige Chen, Sinan Wang, Yida Tao, and Yepang Liu. 2024. Model-Based GUI Testing for HarmonyOS Apps. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2411–2414.
 - [14] Michael I Gordon, Deokhwan Kim, Jeff H Perkins, Limei Gilham, Nguyen Nguyen, and Martin C Rinard. 2015. Information flow analysis of android applications in droidsafe.. In *NDSS*, Vol. 15. 110.
 - [15] Neville Grech and Yannis Smaragdakis. 2017. P/taint: Unified points-to and taint analysis. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–28.
 - [16] Simon Holm Jensen, Anders Møller, and Peter Thiemann. 2009. Type analysis for JavaScript. In *International Static Analysis Symposium*. Springer, 238–255.
 - [17] Kadiray Karakaya, Stefan Schott, Jonas Klauke, Eric Bodden, Markus Schmidt, Linghui Luo, and Dongjie He. 2024. SootUp: A Redesign of the Soot Static Analysis Framework. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 229–247.
 - [18] Ranjith Krishnamurthy, Goran Piskachev, and Eric Bodden. 2022. To what extent can we analyze Kotlin programs using existing Java taint analysis tools?. In *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 230–235.
 - [19] Li Li, Alexandre Bartel, Tegawendé F Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Outeau, and Patrick McDaniel. 2015. Iccta: Detecting inter-component privacy leaks in android apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 280–291.
 - [20] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé Bissyande, et al. 2023. Software engineering for openharmony: A research roadmap. *Comput. Surveys* (2023).
 - [21] Wei Li, Dongjie He, Yujiang Gui, Wenguang Chen, and Jingling Xue. 2024. A context-sensitive pointer analysis framework for Rust and its application to call graph construction. In *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*. 60–72.
 - [22] Zhihao Lin, Mingyi Zhou, Wei Ma, Chi Chen, Yun Yang, Jun Wang, Chunming Hu, and Li Li. 2025. HapRepair: Learn to Repair OpenHarmony Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 319–330.
 - [23] Farong Liu, Mingyi Zhou, Yakun Zhang, Ting Su, Bo Sun, Jacques Klein, Xiang Gao, and Li Li. 2025. HapTest: The Dynamic Analysis Framework for OpenHarmony. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 422–431.
 - [24] Runlin Liu, Yuhang Lin, Yunge Hu, Zhe Zhang, and Xiang Gao. 2024. LLM-Based Java Concurrent Program to ArkTS Converter. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2403–2406.
 - [25] Benjamin Livshits. 2006. *Improving software security with precise static and runtime analysis*. Vol. 67. Citeseer.
 - [26] Long Lu, Zhichun Li, Zhenyu Wu, Wenke Lee, and Guofei Jiang. 2012. Chex: statically vetting android apps for component hijacking vulnerabilities. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 229–240.
 - [27] Abdalla Wasef Marashdih, Zarul Fitri Zaaba, and Khaled Suwais. 2023. An enhanced static taint analysis approach to detect input validation vulnerability. *Journal of King Saud University-Computer and Information Sciences* 35, 2 (2023), 682–701.
 - [28] Joydeep Mitra and Venkatesh-Prasad Ranganath. 2017. Ghera: A Repository of Android App Vulnerability Benchmarks. In *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering (Toronto, Canada) (PROMISE)*. Association for Computing Machinery, New York, NY, USA, 43–52. doi:10.1145/3127005.3127010
 - [29] Austin Mordahl. 2023. Automatic testing and benchmarking for configurable static analysis tools. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1532–1536.
 - [30] Nomair A Naeem, Ondřej Lhoták, and Jonathan Rodriguez. 2010. Practical extensions to the IFDS algorithm. In *Compiler Construction: 19th International Conference, CC 2010, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2010, Paphos, Cyprus, March 20-28, 2010. Proceedings 19*. Springer, 124–144.
 - [31] Joonyoung Park, Jiyeok Park, Dongjun Youn, and Sukyoung Ryu. 2021. Accelerating JavaScript static analysis via dynamic shortcuts. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1129–1140. doi:10.1145/3468264.3468556
 - [32] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 49–61.
 - [33] Jordan Samhi, Maria Kober, Abdoul Kader Kabore, Steven Arzt, Tegawendé F Bissyandé, and Jacques Klein. 2023. Negative results of fusing code and documentation for learning to accurately identify sensitive source and sink methods: An application to the android

- framework for data leak detection. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 783–794.
- [34] Philipp Dominik Schubert, Ben Hermann, and Eric Bodden. 2019. Phasar: An inter-procedural static analysis framework for c/c++. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 393–410.
- [35] Raphael Spreitzer, Gerald Palfinger, and Stefan Mangard. 2018. Scandroid: Automated side-channel analysis of android apis. In *Proceedings of the 11th ACM Conference on Security & Privacy in Wireless and Mobile Networks*. 224–235.
- [36] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th international conference on compiler construction*. 265–266.
- [37] Tian Tan and Yue Li. 2023. Tai-e: A developer-friendly static analysis framework for Java by harnessing the good designs of classics. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1093–1105.
- [38] Marcos Tileria, Jorge Blasco, and Santanu Kumar Dash. 2024. DocFlow: Extracting taint specifications from software documentation. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [39] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 2010. Soot: A Java bytecode optimization framework. In *CASCON First Decade High Impact Papers*. 214–224.
- [40] Parjanya Vyas, Asim Waheed, Yousra Aafer, and N Asokan. 2023. Auditing framework {APIs} via inferred app-side security specifications. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6061–6077.
- [41] Chao Wang, Ronny Ko, Yue Zhang, Yuqing Yang, and Zhiqiang Lin. 2023. Taintmini: Detecting flow of sensitive data in mini-programs with static taint analysis. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 932–944.
- [42] Jie Wang, Yunguang Wu, Gang Zhou, Yiming Yu, Zhenyu Guo, and Yingfei Xiong. 2020. Scaling static taint analysis to industrial soa applications: A case study at alibaba. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1477–1486.
- [43] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A precise and general inter-component data flow analysis framework for security vetting of android apps. *ACM Transactions on Privacy and Security (TOPS)* 21, 3 (2018), 1–32.
- [44] Junbin Zhang, Yingying Wang, Lina Qiu, and Julia Rubin. 2021. Analyzing android taint analysis tools: FlowDroid, Amandroid, and DroidSafe. *IEEE Transactions on Software Engineering* 48, 10 (2021), 4014–4040.
- [45] Jiazhen Zhao, Yuliang Lu, Xin Wang, Kailong Zhu, and Lu Yu. 2021. WTA: a static taint analysis framework for PHP webshell. *Applied Sciences* 11, 16 (2021), 7763.
- [46] Zexin Zhong, Jiangchao Liu, Diyu Wu, Peng Di, Yulei Sui, Alex X Liu, and John CS Lui. 2023. Scalable compositional static taint analysis for sensitive data tracing on industrial micro-services. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 110–121.